

CHAPTER 9

SECURED PROGRAMMING

Exception Handling

What is Exception?

Indication of a problem that occurs during a program's executing.

Examples of exceptions: out-of-bounds array index, arithmetic overflow, division by zero and invalid function parameters.

What is Exception Handling?

- ✓ **Enables programmers to create applications that can resolve or handle exceptions.**
- ✓ **Allow a program to continue executing as if no problem had been encountered.**
- ✓ **Designed to process synchronous errors, which occur when a statement execute.**
- ✓ **Makes the program more robust – reduce the likelihood that errors will be overlooked.**

Overview

- ✓An exception is a point in the code where something out of the ordinary has happened and the regular flow of the program needs to be interrupted; an exception is not necessarily an error.
- ✓A method which has run into such a case will throw an exception using the throw(ExceptionClass) method.
- ✓When an exception is thrown it must be caught by a catch statement that should sit right after a try statement.

What Is Exception?

```
public class Test{
      public static void main(String str[]){
      int y = 0;
      int x = 1;
      // a division by 0 occurs here.
      int z = x/y;
      System.out.println("after division");
    }
  }
```

The execution of the Test stops and this is caused by the division by zero at - x/y - an exception has been thrown but has not been handled properly.

What Is Exception?

Reasons how exception could occur:

i. User has entered invalid data.
ii. A network connection has been lost in the middle of communication, or JVM has run out of memory.
iii. A field that needs to be opened cannot be found.

How To Handle An Exception?

Try-catch block

```
try
{
  // Section of code which might fail
}
catch (Exception1ThatCanHappen E)
{
  // things to do if this exception was thrown..
}
catch (Exception2ThatCanHappen E)
{
  // things to do if this exception was thrown..
}
finally
 {
  //code executed last in any case
}
```

How To Handle An Exception?

Try-catch block → To trap and handle exceptions

- ✓try the statements that generate the exceptions.
- ✓Within the try block we are free to act as if nothing has or can go wrong.
- ✓Then, within one or more catch blocks, write the program logic that deals with all the special cases.

How To Handle An Exception?

```
public class Test{
   public static void main(String str[]){
      int y = 0;
      int x = 1;
      // try block to "SEE" if an exception occurs
      try{
        int z = x/y;
        System.out.println("after division");
         /* catch clause below handles the ArithmeticException generated by
         the division by zero.*/
      } catch (ArithmeticException ae)
      {System.out.println(" attempt to divide by 0");}
      System.out.println(" after catch ");
  }
}
```

How To Handle An Exception?

Output:

attempt to divide by 0
after catch

the statement - System.out.println("after division") - is NOT executed, once an exception is thrown, the program control moves out of the try block into the catch block.